รายงานสัปดาห์ที 11 (Advanced)
Project Planning & Design — Agentic Document Automator

สรุปเนือหา บันทึกผลแล็บ การแก้ข้อบกพร่อง 2 จุด และผลการรันจริง
โปรเจกต์: agentic-document-automator

---

1. ภาพรวมและผลลัพธ์การเรียนรู้

แล็บนีรวมทักษะจากสัปดาห์ก่อน ๆ เข้าด้วยกัน คือการทำงานกับเอกสาร (PDF/Word) จากแล็บ Basic สัปดาห์ที 11 กับแนวคิด config-driven agent จากแล็บ Advanced สัปดาห์ที 8 และ 10
ผลลัพธ์คือระบบจัดการเอกสารทีกำหนดงานได้จากไฟล์ JSON

- ออกแบบ pipeline งานเอกสารแบบ config-driven ทีเพิม/ลด/สลับงานได้โดยไม่ต้องแก้โค้ด
- สร้างเอกสาร Word จากเทมเพลตด้วยการแทนที placeholder แทนการเขียนเนือหาตายตัวในโค้ด
- ใส่ลายนำลงไฟล์ PDF โดยสร้างชันลายนำด้วย reportlab แล้วซ้อนทับด้วย PyPDF2
- รวมไฟล์ PDF และดึงข้อความออกจากทัง PDF และ Word
- บันทึก audit log ของทุกขันตอนลงไฟล์ JSON เพือตรวจสอบย้อนหลัง

2. สรุปเนือหาบทเรียน

2.1 การสร้างเอกสารจากเทมเพลต

แทนทีจะสร้างเอกสารเปล่าแล้วเขียนเนือหาทีละบรรทัดในโค้ด (แบบทีทำในแล็บ Basic)
วิธีนีใช้ไฟล์เทมเพลต `report_template.docx` ทีออกแบบรูปแบบไว้ล่วงหน้าแล้ว
โดยวางตัวยึดตำแหน่ง (placeholder) เช่น `{{REPORT_TITLE}}` หรือ `{{TABLE_PLACEHOLDER}}` ไว้
จากนันโปรแกรมจะหาและแทนทีด้วยข้อมูลจริงจาก config
ข้อดีคือคนทีไม่เขียนโปรแกรมก็แก้รูปแบบเอกสารได้เองใน Word โดยไม่ต้องแตะโค้ดเลย

2.2 หลักการทำงานของลายน้ำ PDF

PDF ไม่มีคำสัง "ใส่ลายนำ" โดยตรง เทคนิคทีใช้คือสร้าง PDF อีกหน้าหนึงทีมีแต่ข้อความลายนำ (วาดด้วย `reportlab` ลงหน่วยความจำผ่าน `io.BytesIO`) แล้วใช้ `page.merge_page()` ของ PyPDF2 ซ้อนสองหน้าเข้าด้วยกัน ความโปร่งแสงมาจากการกำหนดค่า alpha ในสี เช่น `[1, 0, 0, 0.15]` คือสีแดงความทึบ 15% และหมุนข้อความด้วย `canvas.rotate(55)`

3. สรุปผลการทำแล็บ

#	Task type	สิงทีทำ	ไฟล์ผลลัพธ์
1	generate_word_report	แทนที placeholder ในเทมเพลตด้วยข้อมูลจาก config พร้อมแทรกตารางสรุป 4 แถว	generated_report.docx
2	apply_pdf_watermark	ใส่ลายนำ CONFIDENTIAL สีแดงโปร่งแสง 15% เอียง 55 องศา	watermarked_document.pdf

3	merge_pdfs	รวม dummy_doc1 + dummy_doc2 เป็นไฟล์เดียว	merged_docs.pdf (2 หน้า)
4	extract_word_text	ดึงข้อความจากรายงาน Word ทีเพิ่งสร้าง	extracted_word_text.txt
5	extract_pdf_text	ดึงข้อความจาก PDF ต้นฉบับ	extracted_pdf_text.txt

สังเกตว่า task ที 4 ใช้ไฟล์ที task ที 1 สร้างขึนเป็นอินพุต ซึงแสดงว่าลำดับของ task ใน config มีความสำคัญ ถ้าสลับลำดับกัน task ที 4 จะหาไฟล์ไม่เจอ

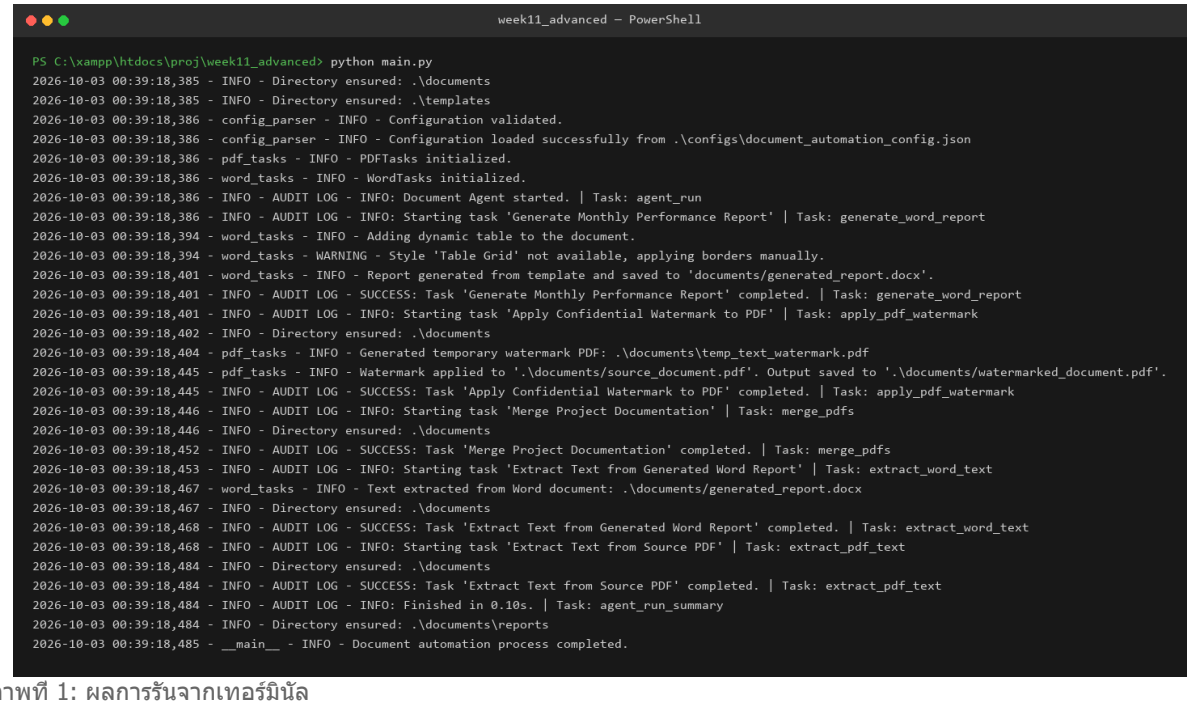
4. ข้อบกพร่องทีพบและการแก้ไข

โค้ดชุดนีนำมาจาก GitHub ของผู้สอนซึงได้แก้ปัญหา relative import ไว้ให้แล้ว (เห็นได้จากคอมเมนต์ในไฟล์) แต่เมือนำมารันจริงยังพบข้อบกพร่องอีก 2 จุด

#	ไฟล์	อาการ	สาเหตุและการแก้ไข
1	word_tasks.py	ตารางในรายงานออกมาไม่มีเส้นขอบเลย พร้อม WARNING: Style 'Table Grid' not available	เทมเพลต report_template.docx มีสไตล์ตารางเพียง 'Normal Table' ไม่มี 'Table Grid' ซึงปกติติดมากับเทมเพลตมาตรฐานของ python-docx โค้ดเดิมดักด้วย try-except แล้วปล่อยผ่านไปเฉย ๆ ตารางจึงไม่มีเส้น → เพิมฟังก์ชัน _apply_table_borders() วาดเส้นขอบเองผ่าน XML (w:tblBorders) เมือไม่มีสไตล์ให้ใช้
2	utils.py	log ถูกพิมพ์ซ้ำ 2 บรรทัด (41 บรรทัด)	basicConfig ใส่ handler ที root แล้ว setup_logging ใส่อีกที logger ลูก เมือ propagate ขึนไปจึงพิมพ์ซ้ำ → เพิม logger.propagate = False ทำให้เหลือ 30 บรรทัด

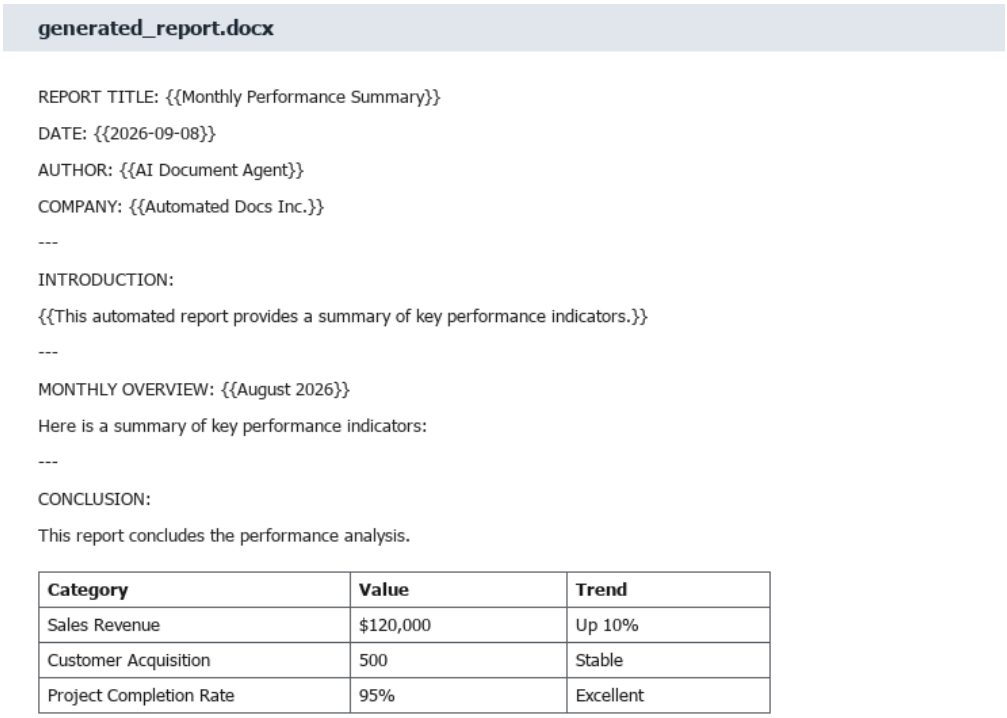
5. ผลการรันจริง (Test Results)

หลังแก้ไข โปรแกรมทำงานจบสมบูรณ์ (exit code 0) ครบทัง 5 tasks และสร้างไฟล์ผลลัพธ์ครบถ้วน



ภาพที 1: ผลการรันจากเทอร์มินัล

5.1 ผลลัพธ์ที่ 1 — generated_report.docx



generated_report.docx

REPORT TITLE: {{Monthly Performance Summary}}

DATE: {{2026-09-08}}

AUTHOR: {{AI Document Agent}}

COMPANY: {{Automated Docs Inc.}}

INTRODUCTION:

{{This automated report provides a summary of key performance indicators.}}

MONTHLY OVERVIEW: {{August 2026}}

Here is a summary of key performance indicators:

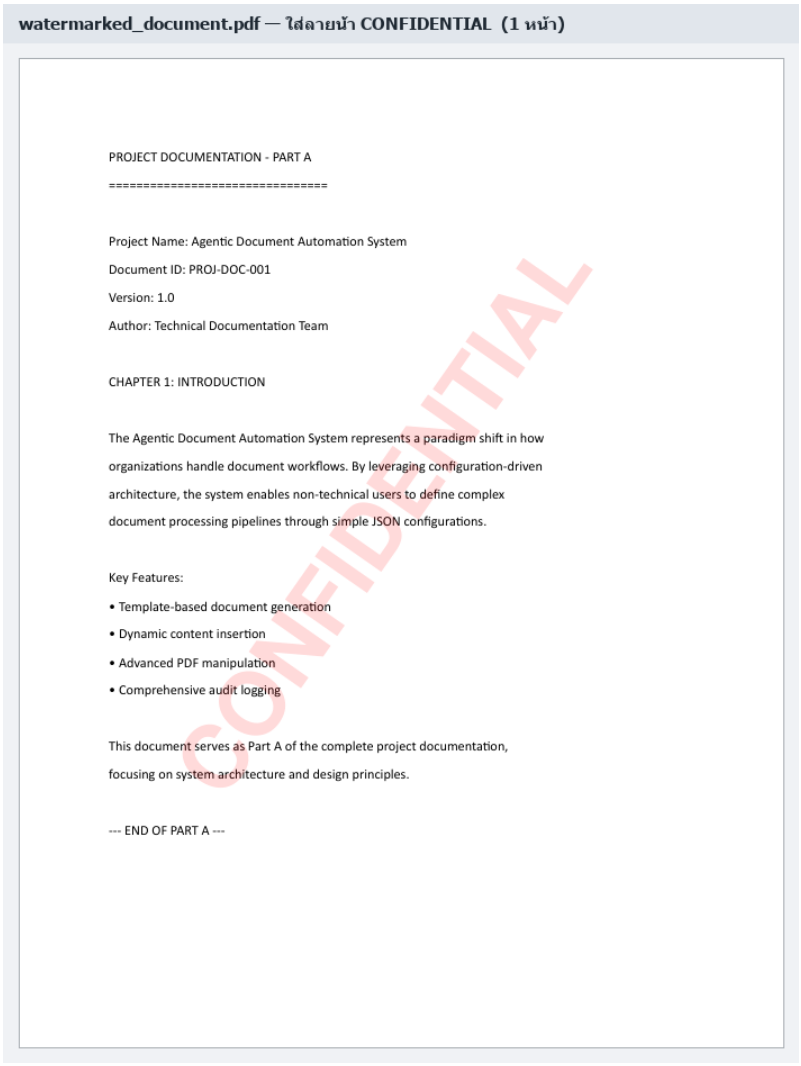
CONCLUSION:

This report concludes the performance analysis.

Category	Value	Trend
Sales Revenue	$120,000	Up 10%
Customer Acquisition	500	Stable
Project Completion Rate	95%	Excellent

ภาพที่ 2: รายงาน Word ที่สร้างจากเทมเพลต ข้อความหัวเรื่องและเนื้อหามาจากฟิลด์ data ใน config ส่วนตารางมาจาก table_data และมีเส้นขอบครบแล้วหลังแก้บักที่ 1

5.2 ผลลัพธ์ที่ 2 — watermarked_document.pdf



ภาพที 3: PDF ทีใส่ลายนำแล้ว เห็นข้อความ CONFIDENTIAL สีแดงจาง ๆ เอียง 55 องศา ซ้อนอยู่ด้านหลังเนือหาเดิมโดยยังอ่านเนือหาได้ปกติ

5.3 ผลลัพธ์ที 3 — merged_docs.pdf



ภาพที่ 4: PDF ที่รวมแล้ว ได้ 2 หน้าตามจำนวนไฟล์ต้นทาง

6. คำถาม-คำตอบสำคัญ

6.1 การใช้เทมเพลตดีกว่าการสร้างเอกสารจากศูนย์อย่างไร?

เพราะแยกรูปแบบออกจากเนื้อหา คนออกแบบเอกสารแก้ฟอนต์ สี โลโก้ หรือหัวกระดาษได้เองใน Word โดยไม่ต้องรู้จัก Python เลย ส่วนโปรแกรมมีหน้าที่แค่เติมข้อมูลลงช่องที่เตรียมไว้ ต่างจากแล็บ Basic ที่ต้องเขียนโค้ดกำหนดทุกอย่าง ซึ่งถ้าจะเปลี่ยนรูปแบบต้องแก้โค้ดและทดสอบใหม่ทุกครั้ง

6.2 ทำไมลายน้ำถึงต้องสร้างเป็น PDF อีกชั้นแล้วซ้อนทับ?

เพราะรูปแบบไฟล์ PDF ถูกออกแบบมาให้เนื้อหา "ตายตัว" ไม่ได้เก็บข้อมูลเป็นโครงสร้างที่แก้ไขง่ายเหมือน Word PyPDF2 จึงแก้ไขเนื้อหาในหน้าโดยตรงไม่ได้ ทำได้แค่จัดการระดับหน้า เช่น เพิ่ม ลบ หมุน หรือซ้อนหน้า การใส่ลายน้ำจึงใช้วิธีสร้างหน้าใหม่ที่มีแต่ลายน้ำ แล้ว merge ทับหน้าเดิม ซึ่งเป็นวิธีมาตรฐานที่ใช้กันทั่วไป

6.3 บทเรียนจากบั๊กเรื่องสไตล์ตาราง

บักนี้น่าสนใจเพราะโค้ดเดิมดักข้อผิดพลาดไว้ถูกต้องแล้ว (try-except ไม่ให้โปรแกรมพัง) และยังเตือนผ่าน WARNING ด้วย แต่ปัญหาคือหลังจากดักแล้วไม่ได้ทำอะไรต่อ ผลคือตารางออกมาไม่มีเส้นขอบ ซึ่งถ้าไม่เปิดไฟล์ดูก็จะไม่รู้ บทเรียนคือ การดักข้อผิดพลาดที่ดีไม่ใช่แค่ไม่ให้โปรแกรมพัง แต่ต้องมีทางสำรองที่ให้ผลลัพธ์ที่ยอมรับได้ด้วย

7. บทสรุป

แล็บนีเป็นการนำสองแนวคิดทีเรียนมาแยกกันมาประกอบเข้าด้วยกัน คือการจัดการเอกสารด้วยโปรแกรม กับการออกแบบระบบทีขับเคลือนด้วย configuration ได้ผลลัพธ์เป็นเครืองมือทีใช้งานได้จริง เช่น ออกรายงานประจำเดือนจากเทมเพลตเดียวกันทุกเดือนโดยเปลียนแค่ข้อมูลใน config หรือใส่ลายน้ำให้เอกสารลับทังโฟลเดอร์ในคำสังเดียว

อ้างอิง: โจทย์ "Week11_ Project Planning & Design: Building an Enhanced Task Manager (LABX Advance)" และซอร์สโค้ดจาก GitHub ของผู้สอน

หมายเหตุ: ภาพเทอร์มินัลสร้างจาก output จริง ส่วนภาพเอกสารเรนเดอร์จากไฟล์ผลลัพธ์จริงทีโปรแกรมสร้างขึน (ไม่ใช่ภาพจับหน้าจอ OS โดยตรง)